

Escape the 90s — CS3383 Project Plan

Team size: 6 developers | **Platform:** Unity (C#), PC build | **Pre-agreed features:** Meta Progression, Side Scroller | **Instructor requirements (BC):** None specified

1. Feature Decomposition (One Feature per Developer)

#	Feature	Owner	Core Idea
1	UI/HUD, Game State & Menu System	TL1	HUD, pause/menu, game-over, overall state flow
2	Procedural Level & Obstacle Generation	TL2	Endless chunk spawning, difficulty scaling
3	Power-Up & Ability System	TL3	Pickups that grant temporary/permanent effects
4	Enemy/Hazard & Combat System	TL4	Spawned threats that damage the player
5	Meta Progression & Persistent Upgrades	TL5	Cross-run currency, save/load, permanent upgrades
6	Player Controller & Side-Scrolling	TL6	Movement, camera scroll, health, death

All six are roughly equal in scope (~52 hours each), independently testable, and each lives entirely inside one developer's *src/*, *tst/*, *prefabs/*, and *docs/* subdirectories.

2. Feature Assignment & Justification

- **TL1 — UI/HUD, Game State & Menu System.** TL1 is Initial Project Lead and creates the initial scene and game loop; owning the top-level state machine (menu → playing → paused → game-over) is a natural fit for the developer who is already responsible for wiring the whole game together and integrating everyone else's public interfaces into it.
 - **TL2 — Procedural Level Generation.** TL2 runs the Week 3 platformer capstone lecture, so this developer already has the deepest hands-on Unity movement/physics experience going into the semester — a natural fit for terrain and obstacle generation.
 - **TL3 — Power-Up & Ability System.** Self-contained effect logic with clear pickup/apply/remove boundaries; low dependency surface makes it a good independent build.
 - **TL4 — Enemy/Hazard & Combat System.** Pairs naturally with TL2's difficulty curve without requiring any shared code — only a read of TL2's public difficulty getter.
 - **TL5 — Meta Progression & Persistent Upgrades.** This is the team's pre-agreed feature, assigned directly to TL5.
 - **TL6 — Player Controller & Side-Scrolling.** TL6 later owns the WebGL/mobile builds; movement, physics, and camera code is exactly the layer most likely to need per-platform tuning (touch input, viewport scaling, frame-rate handling), so having ownership of it early sets TL6 up for that later responsibility.
-

3. Full Feature Specifications

TL1 — UI/HUD, Game State & Menu System

- **Purpose:** Owns all UI screens and the overall game-flow state machine — the shell the rest of the game runs inside.
- **Responsibilities:** HUD display (health/score/currency), pause/resume, game-over + restart, main menu, state transitions.
- **Public interface:**
 - `GameStateManager.CurrentState` (read-only property)
 - `GameStateManager.OnStateChanged` (event)
 - `GameStateManager.RequestPause()` / `RequestResume()` / `RequestRestart()`
 - `HUDController.UpdateScore(int score)` / `UpdateHealth(int hp)`
- **Internal classes:** abstract `GameState`, `MenuState`, `PlayingState`, `PausedState`, `GameOverState`, `GameStateManager`, `HUDController`.
- **Class hierarchy / dynamic binding:** `GameState.Enter()/Update()` are virtual; each state overrides them. Removing `virtual` makes the HUD/pause behavior identical across all states (e.g., HUD never hides on pause).

- **GOF pattern:** State pattern (large) for overall game flow.
- **GRASP:** Controller (*GameStateManager* coordinates flow), Low Coupling (UI only listens to events), Pure Fabrication (*HUDController* separates display from state logic).
- **Dependencies:** Subscribes (read-only, via events) to *OnPlayerDied*, *OnPowerUpCollected*, *OnEnemyDefeated*, and currency updates from the other five features.
- **Testing strategy:** Unit-test state transitions with mocked triggers; boundary tests at game start/end; stress test rapid pause/resume toggling.
- **Completion criteria:** HUD updates correctly in real time; pause/resume/restart all function; game-over screen displays and restarts cleanly.
- **Effort:** Design 9h · Coding 24h · Testing 10h · Documentation 9h = **52h**

TL2 — Procedural Level & Obstacle Generation

- **Purpose:** Generates the endless side-scrolling level ahead of the player and recycles it behind.
- **Responsibilities:** Chunk spawning/pooling, difficulty scaling with distance, obstacle placement, chunk library management.
- **Public interface:**
 - *LevelGenerator.SetDifficulty(float level)*
 - *LevelGenerator.GetCurrentDistance() : float*
 - *LevelGenerator.OnChunkSpawned (event)*
- **Internal classes:** abstract *LevelChunk*, *GroundChunk*, *GapChunk*, *RampChunk*, *ChunkPool*, *LevelGenerator*.
- **Class hierarchy / dynamic binding:** *LevelChunk.Populate()* is virtual; subclasses override it to place gaps/ramps/hazards differently. Removing *virtual* makes every chunk generate as flat ground only.
- **GOF pattern:** Two small patterns — Prototype (cloning chunk templates from the pool) and Façade (*LevelGenerator* as a simple front over pooling/spawning internals).
- **GRASP:** Creator (*LevelGenerator* creates chunks), Pure Fabrication (*ChunkPool*), Protected Variations (new chunk types plug in without touching the spawner).
- **Dependencies:** Reads TL6's *GetPosition()* to know how far ahead to generate (read-only).
- **Testing strategy:** Unit-test pool reuse; boundary tests at distance 0 and max difficulty; stress test spawn rate at high player speed.
- **Completion criteria:** Level generates indefinitely with no visible seams or repeats, difficulty visibly scales with distance.
- **Effort:** Design 10h · Coding 25h · Testing 9h · Documentation 8h = **52h**

TL3 — Power-Up & Ability System

- **Purpose:** Spawns pickups that grant the player temporary or permanent effects.
- **Responsibilities:** Spawning power-up items, detecting pickup, applying/removing effects, managing duration and stacking.
- **Public interface:**
 - `PowerUpManager.SpawnPowerUp(Vector3 position, PowerUpType type)`
 - `PowerUpManager.OnPowerUpCollected` (event)
- **Internal classes:** abstract `PowerUpEffect`, `SpeedBoostEffect`, `ShieldEffect`, `MagnetEffect`, `PowerUpManager`, `PowerUpItem` (MonoBehaviour).
- **Class hierarchy / dynamic binding:** `PowerUpEffect.Apply()` is virtual; each subclass overrides it for a distinct effect. Removing `virtual` makes every pickup apply identical (generic) behavior.
- **GOF pattern:** Two small patterns — Strategy (selecting which effect to apply) and Memento (snapshotting/restoring player stats around temporary effects).
- **GRASP:** Polymorphism (effect subclasses), Indirection (`PowerUpManager` mediates item↔player), Low Coupling.
- **Dependencies:** Calls TL6's `SetSpeedMultiplier()` / `Heal()` — documented interface only.
- **Testing strategy:** Unit-test each effect's apply/remove against a mock target; boundary tests at 0 duration and max stack; stress test many simultaneous power-ups.
- **Completion criteria:** At least 3 visually distinct power-up types function correctly, including stacking behavior.
- **Effort:** Design 9h · Coding 25h · Testing 10h · Documentation 8h = **52h**

TL4 — Enemy/Hazard & Combat System

- **Purpose:** Populates the level with threats that challenge and damage the player.
- **Responsibilities:** Enemy spawning tied to difficulty, enemy movement/attack behavior, collision damage, enemy removal.
- **Public interface:**
 - `EnemySpawner.SetSpawnRate(float rate)`
 - `EnemySpawner.OnEnemyDefeated` (event)
- **Internal classes:** abstract `EnemyBase`, `GroundEnemy`, `FlyingEnemy`, `TurretHazard`, `EnemySpawner`.
- **Class hierarchy / dynamic binding:** `EnemyBase.Move()` is virtual; subclasses override movement/attack patterns. Removing `virtual` makes every enemy move like a ground enemy regardless of type.
- **GOF pattern:** Factory Method (large) — `EnemySpawner` creates the correct enemy subtype.
- **GRASP:** Creator (`EnemySpawner`), Information Expert (`EnemyBase` knows its own threat/attack logic), Protected Variations (new enemy types don't touch spawner code).

- **Dependencies:** Calls TL6's `TakeDamage()` on collision; reads TL2's `GetCurrentDistance()`/difficulty for spawn scaling — both read-only/documented.
- **Testing strategy:** Unit-test each enemy type in isolation; boundary tests at spawn rate 0 and max; stress test many enemies on-screen at once.
- **Completion criteria:** At least 2 enemy/hazard types active, correctly damage the player, spawn rate scales with difficulty.
- **Effort:** Design 10h · Coding 24h · Testing 10h · Documentation 8h = **52h**

TL5 — Meta Progression & Persistent Upgrade System

- **Purpose:** Tracks currency earned per run and lets the player spend it on permanent upgrades between runs.
- **Responsibilities:** Currency tracking, save/load persistence, upgrade-shop logic, applying starting bonuses each run.
- **Public interface:**
 - `MetaProgressionManager.AddCurrency(int amount)`
 - `MetaProgressionManager.GetUpgradeLevel(UpgradeType type) : int`
 - `MetaProgressionManager.PurchaseUpgrade(UpgradeType type)`
 - `MetaProgressionManager.ApplyStartingBonuses(GameObject player)`
- **Internal classes:** abstract `UpgradeNode`, `HealthUpgrade`, `SpeedUpgrade`, `MagnetRangeUpgrade`, `SaveDataManager`, `MetaProgressionManager`.
- **Class hierarchy / dynamic binding:** `UpgradeNode.GetEffectValue()` is virtual; each subclass overrides it. Removing `virtual` makes every upgrade grant an identical generic bonus.
- **GOF pattern:** Two small patterns — Singleton (`SaveDataManager`) and Private Class Data (encapsulating raw save fields so only `SaveDataManager` touches them).
- **GRASP:** Information Expert (`UpgradeNode` knows its own effect), Protected Variations (new upgrade types addable), High Cohesion (`SaveDataManager` does persistence only).
- **Dependencies:** Calls TL6's public setters at run start to apply bonuses — documented interface only.
- **Testing strategy:** Unit-test save/load round-trip with mock data; boundary tests at 0 currency and max upgrade level; stress test rapid purchase spam.
- **Completion criteria:** Currency persists across sessions; at least 3 upgrade types purchasable and correctly applied at run start.
- **Effort:** Design 10h · Coding 24h · Testing 10h · Documentation 8h = **52h**

TL6 — Player Controller & Side-Scrolling System

- **Purpose:** Owns player movement, the scrolling camera, and player health/death — the spine the rest of the game hangs off of.
 - **Responsibilities:** Input handling, movement physics, camera follow/scroll, collision with terrain, triggering game-over on death, exposing player state.
 - **Public interface:**
 - `PlayerHealth.TakeDamage(int amount)`
 - `PlayerHealth.Heal(int amount)`
 - `PlayerController.SetSpeedMultiplier(float mult)`
 - `PlayerController.GetPosition() : Vector3`
 - `PlayerController.OnPlayerDied(event)`
 - **Internal classes:** `PlayerController` (`MonoBehaviour`), `PlayerHealth`, abstract `MovementState`, `RunningState`, `JumpingState`, `DashingState`, `CameraScroller`.
 - **Class hierarchy / dynamic binding:** `MovementState` base declares `virtual void Execute()`; `RunningState`/`JumpingState`/`DashingState` override it. Removing `virtual` collapses all three into running behavior only — jump and dash visibly stop working.
 - **GOF pattern:** State pattern (large) for movement states.
 - **GRASP:** Controller (`PlayerController` coordinates state), Information Expert (`PlayerHealth` owns its own HP), Polymorphism (movement states), Low Coupling (only events/getters exposed).
 - **Dependencies:** None required; other features read `GetPosition()`/call setters.
 - **Testing strategy:** Unit-test state transitions with mocked input; boundary tests at 0 HP and max HP; stress test rapid state switching.
 - **Completion criteria:** Player runs/jumps/dashes smoothly, dies at 0 HP and fires the event, camera scrolls endlessly without stutter.
 - **Effort:** Design 10h · Coding 24h · Testing 10h · Documentation 8h = **52h**
-

4. Architectural Overview

UI/HUD & Game State (TL1) is the shell everything else runs inside — it owns the top-level state machine (menu → playing → paused → game-over), sets up the initial scene, and reacts to events fired by every other feature without ever calling into their internals.

Level Generation (TL2) runs independently of the player except for one read-only position check, spawning and recycling chunks ahead of and behind the player to create the illusion of an endless world, and exposing a distance/difficulty value that other systems can key off of.

Power-Ups (TL3) sit in the world as pickups placed by the level (or spawned independently) and, on collection, apply short bursts of effect onto the player through TL6's public setters, then clean themselves up via the Memento-based restore.

Enemies/Hazards (TL4) spawn based on TL2's difficulty signal and interact with the player only through TL6's `TakeDamage()` — they have no awareness of power-ups or the UI.

Meta Progression (TL5) operates almost entirely out-of-band, at the start and end of a run: it loads/saves persistent data and applies starting bonuses to the player through TL6's interface, with no in-run dependency on any other feature.

Player Controller (TL6) is the physical center of the game — every other system either reads its state (position) or calls into it (damage, speed, healing) through narrow, documented methods, and it never reaches into anyone else's code.

5. MVP Core

TL1 — UI/HUD, Game State & Menu System is the MVP core. As Initial Project Lead, TL1 creates the initial scene and game loop that every other feature plugs into; the `GameStateManager` is the thing that starts a run, calls into the player controller, level generator, power-ups, enemies, and meta progression through their public interfaces, and ends the run. A build with just the state machine and an empty `PlayingState` is already the skeleton the MVP is built on — every other feature is wired in as a layer the state machine drives, not a prerequisite for it to exist.

6. Low Coupling / High Cohesion via GRASP

Coupling stays low because every cross-feature interaction is a single documented method call or event subscription — never a shared field, shared script, or edited foreign directory. TL3 and TL5 only ever call outward into TL6's setters (Information Expert lets TL6 remain the sole authority on its own health/speed state). TL4 and TL3 both key off TL2's difficulty/distance getters rather than each other, avoiding an enemy↔power-up dependency entirely. TL1 achieves the loosest coupling of all: it never calls into any other feature's internals directly, only listens to their events and drives the top-level state machine (Indirection), which means UI/state code can change freely without risking any other developer's feature.

Cohesion stays high because each feature's internal classes serve one purpose: TL2's chunk classes only generate terrain, TL5's `SaveDataManager` only persists data (Pure Fabrication keeps it from being tangled into gameplay logic), and TL1's `HUDController` only draws — it

never decides game logic, that's left to *GameStateManager* (Controller). Protected Variations shows up in TL2, TL4, and TL5 alike: new chunk types, enemy types, and upgrade types can all be added as new subclasses without modifying the spawner/manager that creates them.

7. Initial Gantt Chart (Weeks 4–12)

Dev	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8	Wk 9	Wk 10	Wk 11	Wk 12
TL1	State manager skeleton	SA prep, state diagram	Diagrams, HUD tests	HUD wired to events	Midterm, light bugfix	State pattern cleanup	Integrate all interfaces	CI/CD fixes, docs	Oral exam prep, polish
TL2	Chunk pool skeleton	SA prep, chunk diagram	Diagrams, obstacle types	Difficulty scaling live	Midterm, light bugfix	Prototype/Façade cleanup	Integration support	Performance profiling	Oral exam prep, polish
TL3	Power-up skeleton	SA prep, effect diagram	Diagrams, Strategy setup	Effects functional	Midterm, light bugfix	Memory to snapshot logic	Integration support	Stacking edge cases	Oral exam prep, polish
TL4	Enemy skeleton	SA prep, enemy diagram	Diagrams, spawn tests	Combat functional	Midterm, light bugfix	Factory Method cleanup	Integration support	Balance + polish	Oral exam prep, polish

TL5	Save system skeleton	SA prep, upgrade diagram	Diagrams, save tests	Upgrades functional	Midterm, light bugfix	Singleton refactor	Integration support	Save/load hardening	Oral exam prep, polish
TL6	Player skeleton + movement	SA prep, class diagram	Diagrams due, dash state	Camera scroll polish	Midterm, light bugfix	State pattern refactor	Integration support	Balance + polish	Oral exam prep, polish

8. Risks & Mitigations

1. **Difficulty-signal coupling drift** — TL3/TL4 both key off TL2's difficulty getter; risk they start reading each other's state instead. Mitigation: lock the shared read-only interface at Week 5 SA presentation; no new cross-feature reads after that without a design review.
2. **Endless generation performance degradation** — poor pooling in TL2 could tank frame rate over a long run. Mitigation: profile pooling behavior during Week 6 test-plan work, not just at the end.
3. **Save data corruption** — TL5's persistence layer is the one feature that survives between sessions, so a bad write is the hardest bug to recover from. Mitigation: versioned save schema plus boundary tests (0 currency, corrupted file) from the start.
4. **Uneven effort load** — level generation (TL2) and combat (TL4) have the most moving parts and could balloon past 52 hours. Mitigation: weekly time-log check-ins (Weeks 7, 9, 10, 11, 12) to catch drift early.
5. **Integration bottleneck at TL1** — if public interfaces aren't finalized before Week 10, TL1 can't integrate on schedule. Mitigation: freeze all public method signatures at the Week 5 SA presentation; changes after that require a team sign-off.
6. **Unity scene merge conflicts** — directory ownership prevents source conflicts, but everyone still touches the integration scene. Mitigation: TL1 owns the single integration scene file; other developers work in isolated test scenes and prefabs only.
7. **Forced/artificial GOF pattern usage** — patterns picked to satisfy the requirement rather than the design. Mitigation: each feature's pattern (State, Factory Method, Strategy, Prototype, Façade, Singleton, Memento, Private Class Data) was chosen because it's the natural fit for that system, not shoehorned — worth re-confirming at the Week 7 GRASP/Roast review.

Generated from the CS3383 TL5 AI Project Planning Prompt for "Escape the 90s." Each developer should now use their feature specification (Section 3) to write their individual Champion document.